

Conversione di un'immagine fotografica in una matrice di celle RGB

Algoritmo, motivazioni e ottimizzazione per Excel e Google Fogli

Documento tecnico esplicativo

12 agosto 2026

Obiettivo

Trasformare una fotografia in un foglio elettronico nel quale ogni pixel della versione ridimensionata corrisponde a una cella quadrata. Ogni cella può usare esclusivamente uno dei tre colori primari digitali:  rosso #FF0000,  verde #00FF00,  blu #0000FF. La distribuzione spaziale dei tre colori ricostruisce forme, luminosità e variazioni cromatiche mediante dithering.

1 Risultato e dati principali

La fotografia elaborata misurava 1086×1448 pixel. Per ottenere un documento sufficientemente dettagliato ma ancora esportabile e importabile in Google Fogli, è stata adottata una matrice di

$$543 \times 724 = 393\,132 \text{ celle-pixel.}$$

Il rapporto d'aspetto resta invariato (3 : 4) e ciascuna dimensione lineare è dimezzata. Il numero complessivo di pixel è quindi un quarto dell'immagine di partenza.

Fase	Dimensioni	Funzione
Immagine sorgente	1086×1448	riferimento fotografico completo
Immagine di lavoro	543×724	matrice compatibile e dettagliata
Foglio finale	543×724 celle	un pixel per cella quadrata
Palette	3 colori	RGB primari puri

2 Panoramica dell'algoritmo

La pipeline è composta da sette operazioni:

1. lettura e normalizzazione dell'immagine in RGB;
2. ridimensionamento con filtro Lanczos;
3. inizializzazione della palette primaria;
4. scelta del colore più vicino con distanza euclidea;
5. diffusione dell'errore con Floyd–Steinberg;
6. scrittura della matrice degli indici nel foglio;
7. applicazione di riempimenti statici e verifica del file XLSX.

3 Spiegazione a fronte

Nelle tabelle seguenti, a sinistra è riportata l'operazione algoritmica; a destra la sua funzione e la ragione della scelta.

3.1 1. Lettura e ridimensionamento

Algoritmo / codice

```
image = Image.open(source).convert("RGB")
image = image.resize(
    (543, 724),
    Image.Resampling.LANCZOS
)
```

Spiegazione

La conversione in RGB garantisce tre canali numerici uniformi, ciascuno compreso tra 0 e 255. Il filtro Lanczos riduce la fotografia preservando meglio bordi e dettagli rispetto a un semplice campionamento del pixel più vicino. Il rapporto $543/724 = 3/4$ conserva le proporzioni originali.

Il ridimensionamento può essere interpretato come una ricostruzione filtrata. Per ogni nuovo pixel vengono combinati più pixel vicini dell'immagine sorgente attraverso un kernel sinc finestrato. Ciò limita scalettature e perdita brusca di dettaglio prima della drastica riduzione cromatica.

3.2 2. Definizione della palette

Algoritmo / codice

```
primaries = [
    (255, 0, 0), # rosso
    (0, 255, 0), # verde
    (0, 0, 255) # blu
]
```

Spiegazione

La palette contiene esattamente i primari additivi dello spazio RGB. Non sono ammessi bianco, nero, grigio, ciano, magenta, giallo o tonalità intermedie. I colori percepiti emergono soltanto dalla mescolanza ottica di celle adiacenti.

3.3 3. Scelta del colore primario più vicino

Dato un pixel corrente $\mathbf{p} = (r, g, b)$ e un colore candidato $\mathbf{c}_i$, viene calcolata la distanza euclidea quadratica nello spazio RGB:

$$d_i^2 = (r - c_{i,r})^2 + (g - c_{i,g})^2 + (b - c_{i,b})^2.$$

Il colore assegnato è

$$i^* = \underset{i \in \{R, G, B\}}{\operatorname{argmin}} d_i^2.$$

Algoritmo / codice

```
old = work[y][x]
index = min(
    range(3),
    key=lambda i: sum(
        (old[k] - primaries[i][k]) ** 2
        for k in range(3)
    )
)
new = primaries[index]
```

Spiegazione

Si confronta il pixel con tutti e tre i primari e si sceglie quello geometricamente più vicino. La radice quadrata non viene calcolata: non cambierebbe l'ordine delle distanze e aggiungerebbe lavoro inutile. Senza la fase successiva, questa scelta produrrebbe grandi aree uniformi e perderebbe quasi tutti i chiaroscuri.

3.4 4. Calcolo e diffusione dell'errore

L'errore di quantizzazione è un vettore RGB:

$$\mathbf{e} = \mathbf{p} - \mathbf{c}_{i^*}.$$

Invece di eliminarlo, l'algoritmo lo distribuisce sui pixel non ancora elaborati secondo la matrice Floyd–Steinberg:

$$\begin{array}{ccc} & \boxed{\text{pixel corrente}} & \\ \frac{3}{16} & & \frac{7}{16} \\ & \frac{5}{16} & \frac{1}{16} \end{array}$$

Algoritmo / codice

```
error = [old[k] - new[k] for k in range(3)]

neighbors = (
    ( 1, 0, 7/16),
    (-1, 1, 3/16),
    ( 0, 1, 5/16),
    ( 1, 1, 1/16)
)

for dx, dy, factor in neighbors:
    nx, ny = x + dx, y + dy
    if inside_image(nx, ny):
        work[ny][nx] += error * factor
        work[ny][nx] = clamp(work[ny][nx], 0, 255)
```

Spiegazione

Il pixel appena elaborato non può riprodurre esattamente il colore originale. La differenza residua viene trasferita ai quattro vicini futuri. La somma dei pesi è $7/16 + 3/16 + 5/16 + 1/16 = 1$: l'errore non viene perso, ma ridistribuito. Il vincolo 0–255 impedisce valori RGB non validi. A distanza normale, l'occhio integra il mosaico e percepisce tonalità e luminosità che non esistono nelle singole celle.

3.5 5. Produzione della matrice discreta**Algoritmo / codice**

```
indices[y][x] = index

# Valori possibili:
# 0 -> #FF0000
# 1 -> #00FF00
# 2 -> #0000FF
```

Spiegazione

L'immagine non viene trasferita nel foglio come bitmap incorporata. Si salva invece una matrice numerica di 543 colonne e 724 righe. Ogni elemento identifica il riempimento statico della cella corrispondente. Questa rappresentazione rende ogni pixel realmente selezionabile e modificabile come cella.

3.6 6. Celle quadrate e riempimenti statici**Algoritmo / pseudocodice**

```
set_column_width(3 pixels)
set_row_height(3 pixels)
hide_gridlines()
hide_numeric_values()

for each horizontal run:
    apply_static_fill(run, RGB[index])
```

Spiegazione

Altezza e larghezza sono impostate allo stesso valore, affinché ogni cella appaia quadrata. I numeri 0, 1 e 2 restano memorizzati ma vengono nascosti con un formato numerico vuoto. I colori sono riempimenti permanenti, non regole condizionali: questo evita il foglio bianco osservato durante l'importazione in Google Fogli.

3.7 7. Ottimizzazione per sequenze orizzontali

Applicare lo stile cella per cella richiederebbe 393 132 operazioni. Per ridurre il numero di modifiche, ogni riga è suddivisa in sequenze consecutive dello stesso colore (run-length styling):

```
for y in range(height):
    start = 0
    color = indices[y][0]
    for x in range(1, width + 1):
        if x == width or indices[y][x] != color:
            fill_range(y, start, x - 1, palette[color])
            start = x
        if x < width:
            color = indices[y][x]
```

Una sequenza come

R, R, R, G, G, B, B, B, B

richiede tre operazioni di stile anziché nove. Il contenuto visivo non cambia: cambia soltanto l'efficienza con cui gli stili vengono scritti nel formato XLSX.

4 Compatibilità con Google Fogli

La prima soluzione sperimentata utilizzava formattazione condizionale: il valore numerico della cella determinava dinamicamente il colore. Sebbene valida in Excel, l'importazione in Google Fogli non applicava correttamente il grande insieme di regole e il risultato appariva bianco.

La soluzione finale adotta pertanto:

- riempimenti statici memorizzati direttamente negli stili XLSX;
- soli tre stili cromatici, riutilizzati in tutto il foglio;
- valori numerici nascosti senza cancellarli;
- griglia disattivata;
- dimensioni uniformi delle righe e delle colonne.

Queste scelte riducono le ambiguità d'importazione e rendono l'aspetto indipendente dal motore di calcolo delle regole condizionali.

5 Perché la risoluzione finale è 543×724

Sono state considerate tre scale:

Larghezza	Altezza	Celle	Esito
180	240	43 200	molto stabile, dettaglio limitato
543	724	393 132	alta risoluzione verificata
1086	1448	1 572 528	esportazione non affidabile / memoria esaurita

La matrice finale contiene nove volte le celle della versione 180×240 . Il livello successivo quadruplicherebbe ancora il carico e, nelle prove effettuate, la serializzazione del documento ha superato anche diversi gigabyte di memoria. Inoltre, un foglio con oltre 1,5 milioni di celle staticamente formattate sarebbe più lento da aprire, modificare e importare.

6 Verifiche eseguite

Prima della consegna sono stati controllati:

1. dimensione logica del foglio: intervallo **A1:TW724**;
2. presenza della cella terminale **TW724**;
3. numero totale: 393 132 celle-pixel;
4. integrità ZIP del contenitore XLSX;
5. presenza nei file di stile dei soli codici **FFFF0000**, **FF00FF00** e **FF0000FF**;
6. assenza di colori intermedi nella matrice;
7. completezza dell'inquadratura, dalla croce superiore alla vegetazione inferiore;
8. resa visiva mediante anteprima renderizzata.

7 Complessità computazionale

Indicando con W la larghezza, H l'altezza e $K = 3$ il numero dei colori:

$$T(W, H) = \Theta(W H K) = \Theta(W H),$$

poiché K è costante. La memoria principale necessaria per il buffer RGB e la matrice degli indici è anch'essa

$$M(W, H) = \Theta(W H).$$

La generazione del foglio dipende inoltre dal numero di sequenze cromatiche orizzontali: nel caso peggiore una sequenza per cella, nel caso migliore una sola sequenza per riga.

8 Limiti e possibili miglioramenti

- La distanza euclidea in RGB è semplice, ma non perfettamente uniforme rispetto alla percezione umana. Una variante potrebbe operare in CIELAB e convertire il risultato ai tre primari solo alla fine.
- Floyd–Steinberg produce una trama direzionale. Il percorso serpentino (sinistra-destra, poi destra-sinistra) può ridurre alcuni artefatti.
- Celle più piccole migliorano la fusione ottica ma possono essere arrotondate diversamente da Excel e Google Fogli.
- Una suddivisione in più fogli permetterebbe di conservare la risoluzione integrale, ma non come unica immagine continua su un solo foglio.
- I tre primari puri non includono nero o bianco: le zone scure e chiare sono simulate esclusivamente attraverso frequenze e accostamenti, con inevitabile alterazione cromatica.

9 Pseudocodice completo

```

INPUT: fotografia RGB
OUTPUT: foglio 543 x 724 con celle rosse, verdi o blu

1. I <- converti_in_RGB(fotografia)
2. I <- ridimensiona_Lanczos(I, 543, 724)
3. P <- [(255,0,0), (0,255,0), (0,0,255)]
4. W <- copia flottante di I
5. crea matrice Q[724][543]

6. per y = 0 ... 723:
7.   per x = 0 ... 542:
8.     p <- W[y][x]
9.     i <- indice del colore P[i] piu vicino a p
10.    Q[y][x] <- i
11.    e <- p - P[i]
12.    diffondi 7/16 di e a (x+1, y)
13.    diffondi 3/16 di e a (x-1, y+1)
14.    diffondi 5/16 di e a (x, y+1)
15.    diffondi 1/16 di e a (x+1, y+1)
16.    limita ogni canale modificato all'intervallo [0,255]

17. crea un foglio di 543 colonne e 724 righe
18. imposta righe e colonne alla stessa dimensione
19. nascondi griglia e valori numerici
20. per ogni riga di Q:
21.   raggruppa le celle consecutive con uguale indice
22.   applica a ogni gruppo il riempimento statico P[indice]
23. esporta in XLSX
24. verifica dimensioni, stili, integrita e anteprima

```

10 Conclusione

Il metodo converte una fotografia continua in una composizione discreta di celle, limitata ai soli primari RGB. La riconoscibilità non deriva dalla fedeltà di ogni singolo pixel, ma dalla conservazione statistica e spaziale dell'errore cromatico. Il dithering trasforma quindi un vincolo severo — tre colori soltanto — in una trama capace di suggerire molti più colori e livelli di luminosità. L'uso di riempimenti statici e di una risoluzione verificata rende infine il risultato utilizzabile sia in Excel sia, con maggiore affidabilità, in Google Fogli.